

PROGRAMMATION FONCTIONNELLE 2 : TD8 ENTIERS EN BINAIRE

avril 2026 — Pierre Rousselin

On rappelle la définition du type `positive` qui modélise les entiers naturels non nuls (strictement positifs) écrits en binaire, et les notations suivantes :

```
Inductive positive : Set :=
  | xI (p : positive) | x0 (p : positive) | xH.
Notation "1" := xH : positive_scope.
Notation "p ~ 1" := (xI p) : positive_scope.
Notation "p ~ 0" := (x0 p) : positive_scope.
```

Exercice 1 : Avec les constructeurs

1. Écrire avec les notations ci-dessus, puis en décimal, les entiers suivants :
 - a) `x0 (xI (x0 xH))`
 - b) `xI (xI (xI xH))`
 - c) `x0 (x0 (xI (xI (xI (xI xH))))))`
2. Soient `p` et `q` deux `positive`. Dans chacun des cas suivants, remplacer `???` par une proposition convenable et donner la preuve de l'implication avec des tactiques :
 - a) `p~1 = q~1 -> ???`
 - b) `p~0 = q~1 -> ???`
 - c) `p~0 = 1 -> ???`
 - d) `p~0~1 = q~0~1 -> ???`

--- * ---

Correction de l'exercice 1 :

1. a) 10 ou `1~0~1~0`
 b) 15 ou `1~1~1~1`
 c) 252 ou `1~1~1~1~1~1~1~0~0`
2. En toute rigueur, on peut remplacer les `False` par n'importe quoi, mais bon...

```
Lemma ex12a (p q : positive) : p~1 = q~1 -> p = q.
Proof. intros H; injection H; exact H. Qed.
Lemma ex12b (p q : positive) : p~0 = q~1 -> False.
Proof. intros H. discriminate H. Qed.
Lemma ex12c (p q : positive) : p~0 = 1 -> False.
Proof. intros H. discriminate H. Qed.
Lemma ex12d (p q : positive) : p~0~1 = q~0~1 -> p = q.
Proof. intros H; injection H; exact H. Qed.
```

--- * ---

Exercice 2 : successeur, prédécesseur

1. Rappeler, ou retrouver la définition de la fonction `succ` qui donne le successeur d'un `positive`.
2. Calculer `succ 1~0~1~1` et `succ 1`
3. Définir la fonction `pred` qui donne le prédécesseur d'un `positive` (pour 1, prendre 1 comme prédécesseur).
4. Calculer `pred 1~0`, `pred 1~1~0`. Revenir sur la question précédente si les résultats sont incorrects.
5. Donner la preuve Rocq de :


```
Lemma pred_succ (p : positive) : pred (succ p) = p.
```

 puis une preuve courte de

Lemma succ_inj (p q : positive) : succ p = succ q -> p = q.

6. Que dire de succ (pred p) ? Énoncer un lemme et le prouver.

--- * ---

Correction de l'exercice 2 :

1. **Fixpoint** succ (p : positive) :=
`match p with`
`| xH => x0 xH`
`| x0 q => xI q`
`| xI q => x0 (succ q)`
`end.`
- 2.
3. Faire très attention au cas 2 (1~0) :
Fixpoint pred (p : positive) :=
`match p with`
`| q~1 => q~0`
`| p~0 => match p with 1 => 1 | _ => (pred p)~1 end`
`| 1 => 1`
`end.`
- 4.
5. **Lemma** pred_succ (p : positive) : pred (succ p) = p.
Proof.
`induction p as [p' IH | p' IH |].`
`- simpl; rewrite IH; destruct p'; reflexivity.`
`- reflexivity.`
`- reflexivity.`
Qed.
Lemma succ_inj (p q : positive) : succ p = succ q -> p = q.
Proof.
`intros H.`
`rewrite -[LHS]pred_succ -[RHS]pred_succ H. reflexivity.`
Qed.
6. **Lemma** succ_pred (p : positive) : p <> 1 -> succ (pred p) = p.
Proof.
`intros H. induction p as [p' IH | p' IH |].`
`- reflexivity.`
`- destruct p'; [reflexivity | | reflexivity].`
`simpl; rewrite IH.`
`+ intros H'. discriminate H'.`
`+ reflexivity.`
`- exfalso; apply H; reflexivity.`
Qed.

--- * ---

Exercice 3 : Soustraction

Définir l'opération binaire **sub** pour la soustraction (*subtraction*) sur **positive**. On ne demande ici aucune garantie dans le cas où $p \leq q$.

--- * ---

Correction de l'exercice 3 : J'ai ça pour l'instant qui semble donner de bons résultats, mais je n'ai pas encore écrit de preuves, donc une erreur a pu se glisser :

```

Fixpoint sub (p q : positive) :=
  match p, q with
  | p'~1, q'~1 | p'~0, q'~0 => (sub p' q')~0
  | p'~1, q'~0 => (sub p' q')~1
  | p'~0, q'~1 => let s := sub p' q' in
    match s with
    | 1 => 1
    | _ => (pred s)~1
    end
  | 1, _ => 1
  | _, 1 => pred p
  end.

```

Voici une solution plus évoluée d'Antoine Rozenknop qui compare les entiers en même temps qu'elle calcule leur soustraction.

```

(* retourne (résultat, p <= q, q <= p) *)
Fixpoint su (p q : positive) :=
  match p, q with
  | p'~1, q'~1 | p'~0, q'~0 => match (su p' q') with
    | (_, true, true) => (1, true, true)
    | (_, true, false) => (1, true, false)
    | (s, false, _) => (s~0, false, true)
    end
  | p'~1, q'~0 => match (su p' q') with
    | (_, _, false) => (1, true, false)
    | (s, true, true) => (1, false, true)
    | (s, false, true) => (s~1, false, true)
    end
  | p'~0, q'~1 => match (su p' q') with
    | (_, true, _) => (1, true, false)
    | (s, false, _) => ((pred s)~1, false, true)
    end
  | p~1, 1 => (p~0, false, true)
  | p~0, 1 => (pred (p~0), false, true)
  | 1, 1 => (1, true, true)
  | 1, _ => (1, true, false)
  end.

Definition sub (p q : positive) :=
  match su p q with ( s, i, sup ) => s end.

--- * ---

```